

Práctica 2

Aprendizaje por Refuerzo

Curso 2025-2026

1. Introducción

En esta práctica se va a desarrollar un proyecto personal de Aprendizaje por Refuerzo Profundo (ARP). Para ello, el alumnado va a utilizar la herramienta Gymnasium de OpenAI [3], que es un framework que ofrece distintos escenarios de aprendizaje por refuerzo en forma de entornos, y que es ampliamente utilizada en investigación. Para entrenar políticas capaces de resolver los problemas propuestos, se utilizará la librería de algoritmos de ARP Stable Baselines 3 [2], que es capaz de manejar entornos de OpenAI Gymnasium. Esta librería tiene implementaciones de la gran mayoría de algoritmos de ARP en Pytorch [1] y es altamente configurable.

2. OpenAI Gymnasium

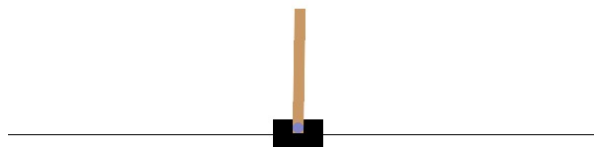
Gymnasium¹ es una API de OpenAI altamente utilizada en la actualidad para investigación de Aprendizaje por Refuerzo. Es considerada estándar en este ámbito y es una versión actualizada del framework anteriormente conocido como OpenAI Gym. Ofrece una gran cantidad de entornos predefinidos de todo tipo, aunque también permite crear nuevos desde cero. Es por ello que es una muy buena herramienta tanto para modelado de nuevos problemas como para probar algoritmos de ARP en problemas ya definidos. En la página web de Gymnasium, en la sección **ENVIRONMENTS**, se nos presentan todos los entornos que vienen definidos en la API, divididos en cinco categorías:

- **Classic Control:** Entornos de control clásico que incluyen problemas comunes en aprendizaje por refuerzo, como el péndulo invertido *CartPole* y el balanceo de un péndulo simple *Pendulum*. Estos entornos suelen usarse para problemas de control básicos.
- **Box2D:** Entornos que utilizan el motor de física Box2D para simular interacciones físicas más complejas. Ejemplos incluyen *LunarLander* y *BipedalWalker*, que permiten trabajar con simulaciones de física realista en entornos de dos dimensiones.
- **Toy Text:** Entornos textuales simples diseñados para resolver problemas de planificación y toma de decisiones sin elementos gráficos complejos. Algunos ejemplos son *FrozenLake* y *Taxi*, donde los agentes realizan acciones basadas en representaciones de texto del estado.
- **MuJoCo:** Entornos de control continuo que utilizan el simulador de física MuJoCo para modelar tareas de control robótico avanzadas. Ejemplos incluyen *Humanoid* y *Ant*, donde los agentes deben aprender a moverse en un espacio tridimensional.
- **Atari:** Entornos basados en juegos de Atari que son populares en el desarrollo de algoritmos de aprendizaje por refuerzo profundo. Estos entornos, como *Breakout*, *Pac-Man*, *Pong* y *SpaceInvaders*, ofrecen problemas de control visual y secuencial, y son desafiantes debido a sus grandes espacios de observación y acción.

¹<https://gymnasium.farama.org/>

2.1. Descripción de los entornos

Podemos acceder, para cada entorno, a su especificación concreta. Pongamos como ejemplo el entorno de péndulo invertido *CartPole*². Cada entorno tiene una descripción detallada de su funcionamiento, de su espacio de acciones y observaciones, de la función de reward y de los estados iniciales y objetivo. Para el caso del *CartPole*, lo que queremos es mantener un péndulo en equilibrio el máximo de tiempo posible.



A continuación, se nos presenta en una breve tabla un resumen inicial del entorno. Nos indica información de cómo es su espacio de acciones y su espacio de observaciones. Además, nos indica el identificador que la API asigna a este entorno, en nuestro caso, **CartPole-v1** (las mayúsculas son importantes).

Action Space	<code>Discrete(2)</code>
Observation Space	<code>Box([-4.8 -inf -0.41887903 -inf], [4.8 inf 0.41887903 inf], (4,), float32)</code>
import	<code>gymnasium.make("CartPole-v1")</code>

Tenemos toda una serie de espacios en Gymnasium que pueden definir los espacios de acciones y de observaciones (o estados). Concretamente, en la descripción de la API, se nos explica todos los tipos que hay definidos (aunque se pueden también definir espacios originales, no es recomendable)³. En esta práctica nos centraremos en dos: el espacio **Discrete** y el espacio **Box**. Como vemos en la Figura 1, el *CartPole* tiene un espacio de acciones discreto, con dos acciones. Si observamos la descripción del espacio de acciones, vemos que ambas acciones corresponden a mover la base del péndulo (la caja negra) a la derecha o a la izquierda.

Action Space

The action is a `ndarray` with shape `(1,)` which can take values `{0, 1}` indicating the direction of the fixed force the cart is pushed with.

- 0: Push cart to the left
- 1: Push cart to the right

Note: The velocity that is reduced or increased by the applied force is not fixed and it depends on the angle the pole is pointing. The center of gravity of the pole varies the amount of energy needed to move the cart underneath it

Como observamos, todos los detalles físicos que funcionan internamente están completamente ajenos al usuario (obsérvese la nota de la imagen anterior).

²https://gymnasium.farama.org/environments/classic_control/cart_pole/

³<https://gymnasium.farama.org/api/spaces/>

Por otro lado, vemos que el espacio de observaciones en el caso de *CartPole* es una **Box** de cuatro dimensiones (tercer parámetro). El espacio **Box** es, a diferencia de **Discrete**, un espacio continuo (acotado o no acotado). Así pues, el espacio de observaciones del *CartPole* consiste de cuatro variables que toman valores en los intervalos definidos por el primer y segundo parámetro. El primer parámetro define los límites inferiores, y el segundo los superiores. De esta manera, podemos observar la descripción del espacio de observaciones:

Observation Space

The observation is a `ndarray` with shape `(4,)` with the values corresponding to the following positions and velocities:

Num	Observation	Min	Max
0	Cart Position	-4.8	4.8
1	Cart Velocity	-Inf	Inf
2	Pole Angle	~ -0.418 rad (-24°)	~ 0.418 rad (24°)
3	Pole Angular Velocity	-Inf	Inf

Note: While the ranges above denote the possible values for observation space of each element, it is not reflective of the allowed values of the state space in an unterminated episode. Particularly:

- The cart x-position (index 0) can be take values between `(-4.8, 4.8)`, but the episode terminates if the cart leaves the `(-2.4, 2.4)` range.
- The pole angle can be observed between `(-.418, .418)` radians (or $\pm 24^\circ$), but the episode terminates if the pole angle is not in the range `(-.2095, .2095)` (or $\pm 12^\circ$)

De esta manera los estados en este entorno se definen con cuatro variables, que representan respectivamente la posición de la caja negra, su velocidad, y el ángulo y velocidad angular del péndulo que debe permanecer en equilibrio. Los rangos vienen especificados en la tabla.

Una vez se ha comprendido qué representa tanto el espacio de observaciones como el espacio de acciones del *CartPole*, debemos conocer cuál es el objetivo de este dominio. Para este ejemplo, el objetivo es mantener el péndulo el mayor tiempo posible en equilibrio, lo cual se puede inferir de la definición de la función de recompensa, en la cual se especifica que se tiene un reward +1 por cada paso.

Rewards

Since the goal is to keep the pole upright for as long as possible, by default, a reward of `+1` is given for every step taken, including the termination step. The default reward threshold is 500 for v1 and 200 for v0 due to the time limit on the environment.

If `sutton_barto_reward=True`, then a reward of `0` is awarded for every non-terminating step and `-1` for the terminating step. As a result, the reward threshold is 0 for v0 and v1.

También se especifica el estado inicial del problema, que es una inicialización aleatoria dentro de un rango, y la condición de parada, que es o bien que la caja se salga del rango de la pantalla, o bien que el péndulo se caiga al suelo, o bien que pasen un total de 500 pasos.

Starting State

All observations are assigned a uniformly random value in `(-0.05, 0.05)`

Episode End

The episode ends if any one of the following occurs:

1. Termination: Pole Angle is greater than $\pm 12^\circ$
2. Termination: Cart Position is greater than ± 2.4 (center of the cart reaches the edge of the display)
3. Truncation: Episode length is greater than 500 (200 for v0)

Adicionalmente, los entornos pueden inicializarse con parámetros extra que implicarían cambios en cualquiera de estas características que hemos identificado anteriormente. Como se observa, los entornos están definidos para que sean completamente amigables de cara al usuario (o al agente de Aprendizaje por Refuerzo). En el caso concreto del *CartPole*, no nos tenemos que preocupar de las físicas del problema, solo de mover la caja a la izquierda o a la derecha.

2.2. Ejemplo de uso de la API

Aunque vamos a utilizar la API con Stable Baselines 3, también es posible programar directamente utilizando métodos disponibles en el entorno. Por ejemplo, el siguiente código utiliza una política aleatoria para navegar por el entorno de *CartPole*.

```
1 import gymnasium as gym
2
3 # Inicializar el entorno
4 env = gym.make("CartPole-v1", render_mode="rgb_array")
5
6 # Reset del entorno para generar la primera observacion
7 # con la distribucion de starting state
8 observation, info = env.reset(seed=42)
9 for _ in range(1000):
10     # Devuelve una accion aleatoria
11     action = env.action_space.sample()
12
13     # Ejecuta un paso en el entorno con la accion que hemos generado
14     # Recibo el siguiente estado, el reward, y si el episodio ha sido
15     # terminado/truncado , junto con informacion adicional (dependiente de entorno)
16     observation, reward, terminated, truncated, info = env.step(action)
17
18     # Si el episodio ha sido terminado o truncado, podemos resetear para empezar un
19     # nuevo episodio
20     if terminated or truncated:
21         observation, info = env.reset()
22
23 env.close()
```

En el Notebook Python `ejemplo_cartpole.ipynb` tenemos un código sencillo que implementa una política aleatoria para *CartPole* (de igual forma que este código) y que guarda un vídeo de lo que va ocurriendo. Esto se consigue con la función **render**, también implementada en Gymnasium para todos los entornos, que nos devuelve una imagen a demanda del estado actual del problema. Si generamos el vídeo y lo vemos, observamos que el péndulo se cae todo el rato porque estamos tomando decisiones aleatorias.

3. Stable Baselines 3

Stable Baselines 3⁴ (SB3) es una biblioteca en Python desarrollada para simplificar el uso de algoritmos de aprendizaje por refuerzo y proporcionar implementaciones eficientes y fáciles de usar de varios algoritmos de RL avanzados. Construida sobre PyTorch, Stable Baselines3 es una versión moderna y optimizada de la biblioteca Stable Baselines, y está diseñada específicamente para la investigación, experimentación y desarrollo de modelos de aprendizaje por refuerzo en entornos como OpenAI Gymnasium y otros entornos compatibles con la interfaz Gym. Es por ello que esta biblioteca es de gran interés para el desarrollo de esta práctica, teniendo además un gran número de algoritmos implementados:

Name	Box	Discrete	MultiDiscrete	MultiBinary	Multi Processing
ARS ¹	✓	✓	✗	✗	✓
A2C	✓	✓	✓	✓	✓
CrossQ ¹	✓	✗	✗	✗	✓
DDPG	✓	✗	✗	✗	✓
DQN	✗	✓	✗	✗	✓
HER	✓	✓	✗	✗	✓
PPO	✓	✓	✓	✓	✓
QR-DQN ¹	✗	✓	✗	✗	✓
RecurrentPPO ¹	✓	✓	✓	✓	✓
SAC	✓	✗	✗	✗	✓
TD3	✓	✗	✗	✗	✓
TQC ¹	✓	✗	✗	✗	✓
TRPO ¹	✓	✓	✓	✓	✓
Maskable PPO ¹	✗	✓	✓	✓	✓

Vemos que algunos algoritmos solamente soportan tipos concretos de espacios de acciones. Por ejemplo, el algoritmo Deep Q-Network (DQN) (tal y como se ha visto en teoría) solamente admite acciones discretas, mientras que Proximal Policy Optimization (PPO) admite acciones tanto discretas como continuas. Esto lo representa refiriéndose directamente a los espacios **Box** y **Discrete** de OpenAI Gymnasium. El entrenamiento de una política para un entorno OpenAI Gymnasium es muy sencillo utilizando SB3. Por ejemplo, para el *CartPole*, podemos entrenar una política de la siguiente manera:

```
1 import gymnasium as gym
2 from stable_baselines3 import DQN
3
4 # Crear el entorno
5 env = gym.make("CartPole-v1", render_mode="rgb_array")
6
7 # Entrenar el modelo DQN por un total de diez mil pasos
8 model = DQN("MlpPolicy", env, verbose=1)
9 model.learn(total_timesteps=10000)
```

⁴<https://stable-baselines3.readthedocs.io>

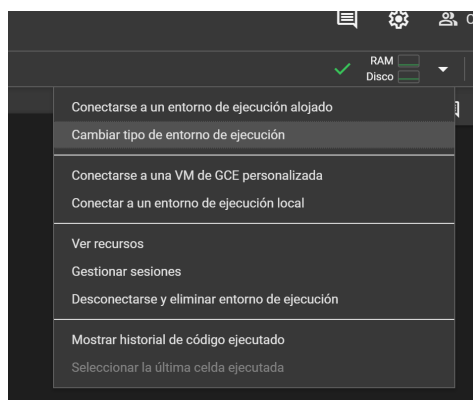
Y luego la podemos utilizar directamente en el entorno para observar cómo se comporta. Donde antes teníamos la acción aleatoria, ahora le decimos al modelo que prediga la siguiente acción a tomar:

```
1 # Reset del entorno
2 observation, info = env.reset(seed=42)
3
4 for _ in range(1000):
5     # Generar la accion usando el modelo entrenado en lugar de una politica aleatoria
6     action, _ = model.predict(observation, deterministic=True)
7
8     # Ejecutar un paso en el entorno con la accion predicha
9     observation, reward, terminated, truncated, info = env.step(action)
10
11     # Si el episodio ha sido terminado o truncado, reiniciar el entorno
12     if terminated or truncated:
13         observation, info = env.reset()
14
15 # Cerrar el entorno
16 env.close()
```

Cada modelo es altamente modificable, con un gran número de parámetros que podemos definir a la hora de crear el modelo. Por, ejemplo, para DQN⁵, algunos de los más relevantes son:

- **learning_rate**: El valor de factor de aprendizaje, de 0 a 1.
- **buffer_size**: El tamaño del replay buffer.
- **gamma**: El factor de descuento
- **train_freq**: La frecuencia con la que la target network se actualiza.

En el Notebook Python `SB3_cartpole.ipynb` tenemos un código sencillo que utiliza SB3 para entrenar una política con DQN para el entorno *CartPole*, generando posteriormente un vídeo de cómo esta política se comporta. Vemos que, en este caso, el péndulo sí consigue estabilizarse sin problema. Para esta práctica, recomendamos utilizar Google Colab⁶. Únicamente necesitamos una cuenta de Google y podremos abrir los archivos `.ipynb` sin problema. Recomendamos, además, utilizar GPU en Colab de la siguiente manera para acelerar cálculos. Haremos click a la flecha al lado de RAM/Disco y cambiaremos el tipo de entorno de ejecución a GPU (no usar TPU).



⁵<https://stable-baselines3.readthedocs.io/en/master/modules/dqn.html>

⁶<https://colab.research.google.com/>

4. Desarrollo de la práctica

Para esta práctica, se responderá a varios ejercicios en forma de PDF. Adicionalmente, **se deberá realizar un vídeo de máximo seis minutos** en el que el alumnado deberá grabarse explicando los objetivos conseguidos, apoyándose de gráficas, vídeos generados de entornos, celdas de código y todo lo que se considere. Los diferentes ejercicios se detallan a continuación:

4.1. Optimización de CartPole (1,5 ptos)

Se debe conseguir, de la forma que se considere, hacer que el péndulo quede en equilibrio los 500 pasos del problema. En este ejercicio se valorará la explicación de por qué, con lo que se ha realizado, el péndulo consigue quedar en equilibrio los 500 pasos y su posterior **justificación**.

4.2. Entrenamiento para un problema de Box2D (4 ptos)

Se debe escoger uno de los tres entornos de la familia Box2D⁷. Para dar más variedad, distinguiremos entre las versiones discretas y continuas. Las opciones son:

- **Lunar Lander (Discrete):** Aterrizaje lunar. Versión por defecto con acciones discretas (motores on/off).
- **Lunar Lander (Continuous):** Aterrizaje lunar con control continuo de potencia. Requiere activar el argumento `continuous=True`.
- **Bipedal Walker (Continuous):** Control de locomoción de un robot bípedo.
- **Car Racing (Continuous):** Conducción de vehículo desde vista superior (píxeles). Versión por defecto con 3 acciones continuas (volante, gas, freno). **Nota:** Al ser observación de imagen, requiere usar políticas convolucionales (`CnnPolicy`).
- **Car Racing (Discrete):** Conducción de vehículo desde vista superior (píxeles). Versión con 5 acciones discretas. Requiere activar `continuous=False`. **Nota:** Al igual que la anterior, requiere `CnnPolicy`.

Observad en la documentación que hay que instalar dos dependencias extras (en el Notebook, se pueden poner al lado de la instalación de Gymnasium). El alumno deberá escoger 3 algoritmos disponibles en SB3 (justificando su elección teniendo en cuenta el entorno escogido) y explicar:

- Descripción del entorno utilizado.
- Descripción y justificación de la elección de los algoritmos.
- Elección de hiperparámetros y evolución del entrenamiento con gráficas/tablas.
- Evaluación de las políticas entrenadas.
- Elección de política/algoritmo/hiperparámetros final.

En este ejercicio se valorará la exposición clara de las estrategias que se han utilizado para entrenar las diferentes políticas, apoyándose de recursos tales como gráficas o tablas, y de su evaluación. También se valorará favorablemente la elección razonada de una política final.

⁷<https://gymnasium.farama.org/environments/box2d/>

4.3. Proyecto libre (2,5 ptos)

En este último apartado deberéis desarrollar un proyecto libre de Aprendizaje por Refuerzo. Esto incluye, entre otras cosas, entrenamientos satisfactorios para cualquier entorno de MuJoCo⁸ (robótica) o Atari⁹ (videojuegos). El alumno es libre de buscar cualquier entorno de Gymnasium que vea interesante, esté o no esté en la librería original, así como de implementar cualquier entorno que le interese. Hay muchísimos entornos de cualquier tipo implementados en GitHub, por ejemplo, para videojuegos tales como League Of Legends¹⁰, videojuegos retro¹¹, etcétera. Este apartado **EXCLUYE**:

- Entrenamientos para entornos de Classic Control, Toy Text o Box2D.
- Algoritmos sencillos vistos en clase o en otras prácticas como Q-Learning.

En este ejercicio se valorará, sobre todo, la originalidad y la buena presentación de los resultados. Se deberá explicar claramente qué entorno se ha decidido utilizar y por qué, **detallando su funcionamiento**. Esto es imprescindible para obtener la máxima puntuación. Se esperan resultados aceptables para entornos complejos, no siendo necesario tener una performance impecable.

4.4. Calidad de la entrega (2 ptos)

Se evaluará la capacidad de comunicación y síntesis de los resultados obtenidos en el conjunto de toda la práctica:

- **Memoria (1 pto):** Se valorará la claridad expositiva, la estructura, la calidad del contenido y, especialmente, la **justificación razonada de las decisiones tomadas**.
- **Vídeo (1 pto):** Máximo 6 minutos. Debe tener un enfoque visual y comparativo. Se espera que el alumno muestre y analice gráficas de rendimiento, tablas comparativas y clips de los agentes actuando, **evitando la mera lectura del código o del documento**.

Referencias

- [1] PASZKE, A., GROSS, S., MASSA, F., LERER, A., BRADBURY, J., CHANAN, G., KILLEEN, T., LIN, Z., GIMELSHEIN, N., ANTIGA, L., DESMAISON, A., KOPF, A., YANG, E., DEVITO, Z., RAISON, M., TEJANI, A., CHILAMKURTHY, S., STEINER, B., FANG, L., BAI, J., AND CHINTALA, S. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32*. Curran Associates, Inc., 2019, pp. 8024–8035.
- [2] RAFFIN, A., HILL, A., GLEAVE, A., KANERVISTO, A., ERNESTUS, M., AND DORMANN, N. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research* 22, 268 (2021), 1–8.
- [3] TOWERS, M., KWIATKOWSKI, A., TERRY, J., BALIS, J. U., DE COLA, G., DELEU, T., GOULÃO, M., KALLINTERIS, A., KRIMMEL, M., KG, A., ET AL. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032* (2024).

⁸<https://gymnasium.farama.org/environments/mujoco/>

⁹<https://ale.farama.org/environments/>

¹⁰<https://github.com/MiscellaneousStuff/lolegym>

¹¹<https://openai.com/index/gym-retro/>